

Elveflow User Guide

SDK SOFTWARE DEVELOPMENT KIT

DOCUMENT REF: UGSDK-Cpp-250429



SYMBOLS USED IN THIS DOCUMENT



IMPORTANT INFORMATION. Disregarding this information may increase the risk of: damage to the equipment, personal injuries, and impact your user experience.



HELPFUL INFORMATION. This information facilitates the use of the instrument and contributes to its optimal performance.



ADDITIONAL INFORMATION. Available on Elveflow website or from your Elveflow representative.

READ THIS MANUAL CAREFULLY BEFORE USING THE SOFTWARE



This manual must be read by every person who is or will be responsible for using the Elveflow software development kit (SDK).

Due to the continual development of the products, the content of this manual may not correspond to the new software. Therefore, we retain the right to make adjustments without prior notification.

Important SDK safety notices:

1. The SDK gives the user complete control over Elveflow products. Beware of pressure limits for containers, chips and other parts of your setup. They might be damaged if the pressure applied is too high.
2. Use a computer with enough power to avoid software freezing.

If these conditions are not **RESPECTED**, the user is exposed to dangerous situations and the instrument can undergo permanent damage. Elveflow and its partners cannot be held responsible for any damage related to the misuse of the instruments.

Table of contents

Getting started	4
Before starting	5
Important remarks	5
DLL programming:	6
Introduction	6
Description of SDK functions for each instrument:	7
OB1:	7
Normal workflow	7
OB1_Initialization	8
OB1_Destructor	8
OB1_Add_Sens	8
OB1_Get_Data	9
OB1_Get_All_Data	9
OB1_Set_Press	10
OB1_Set_Sensor	10
OB1_Get_Trig	10
OB1_Set_Trig	10
OB1_Calib	11
OB1_Reset_Digit_Sens (OB1 MK3+)	11
OB1_Reset_Instr (OB1 MK3+)	11
OB1_Calib_Load	11
OB1_Calib_Save	12
Elveflow_EXAMPLE_PID	12
MSRD:	13
M_S_R_D_Initialization	13
M_S_R_D_Destructor	13
M_S_R_D_Add_Sens	14
M_S_R_D_Get_Data	14
M_S_R_D_Get_Trig (MSR V3)	14
M_S_R_D_Set_Trig (MSR V3)	15
M_S_R_D_Set_Filt	15
M_S_R_D_Reset_Instr (MSR V2)	15
M_S_R_D_Reset_Sens (MSR V2)	15
BFS:	16
BFS_Initialization	16

BFS_Destroyor	16
BFS_Get_Data	16
BFS_Zero	17
BFS_Set_Filter	17
BFS_Set_Params	17
MUX D-R-I (DISTRIBUTION, DISTRIBUTOR, RECIRCULATION or INJECTION):	18
MUX_DRI_Initialization	18
MUX_DRI_Destroyor	18
MUX_DRI_Set_Valve	18
MUX_DRI_Get_Valve	19
MUX_DRI_Send_Command (MUX Distribution 12 & MUX Recirculation 6)	19
Other MUX Series:	20
MUX_Initialization	20
MUX_Destroyor	20
MUX_Get_Trig	20
MUX_Set_Trig	21
MUX_Set_all_valves (MUX FLOW SWITCH)	21
MUX_Set_indiv_valve(MUX CROSS CHIP)	21
MUX_Wire_Set_all_valves (MUX WIRE instrument)	22
MUX_Set_valves_Type (MUX WIRE V3 instrument)	22
MUX_Get_valves_Type (MUX WIRE V3 instrument)	22
Remote PID:	24
PID_Add_Remote	25
PID_Set_Running_Remote	25
PID_Set_Params_Remote	25
Appendix	25
Error handling:	26
List of constants, prototypes and description (for C++, MATLAB and Python):	27
Z_regulator_type:	27
Z_sensor_type:	27
Z_Sensor_digit_analog:	27
Z_Sensor_FSD_Calib:	27
Z_D_F_S_Resolution:	27
Z_MUX_DRI_Rotation:	27
Z_MUX_DRI_Action:	28

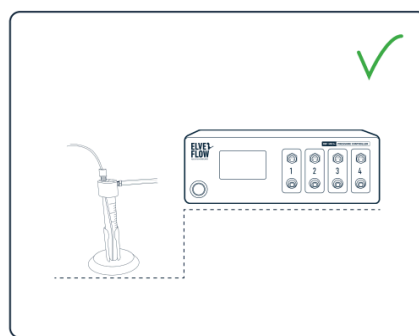
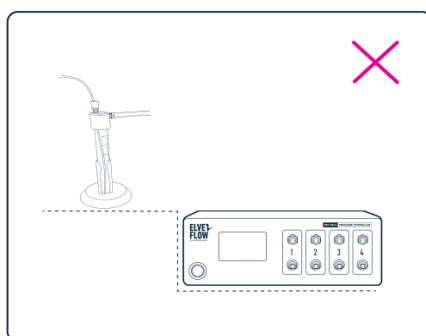
Getting started

Elveflow proposes a standard development kit for LabVIEW, C++, Python and MATLAB

The following sections will guide you through the steps to add a new instrument or sensor, explore its basic and advanced features and use it with other instruments to automate your experiment.

Before starting

To prevent backflow in the pressure regulator, always place liquid reservoirs under the instrument.



Important remarks

For all programming languages:

- For 32 bit DLL users : install the ESI software before using the DLL
- For 64 bit DLL users : install the 64 bit drivers located in the SDK folder
- If MUX Distribution/Distributor/Recirculation/Injection, BFS or OB1 MK4 are used, FTDI drivers are required (<http://www.ftdichip.com/Drivers/D2XX.htm>). You can find these drivers in the same folder the ESI is installed. Default location would be C:\Program Files (x86)\Elvesys\driver (look for driver_MUX_distAndBFS.exe).
- Do not simultaneously use the ESI software and the SDK, some conflict would occur.
- For X64 libraries running on an AMD computer, the impossibility of communicating with instruments has been occasionally reported. Contact us if it is the case. The only fix found so far is to set the environment variable **MKL_DEBUG_CPU_TYPE** to 4.



DLL programming:

Introduction

For C++, MATLAB, and Python programming languages, two C++ DLL libraries common to all languages are available. One for x64 and one for x32 operating systems (DLL32 and DLL64 folders). These libraries (Elveflow32.dll and Elveflow64.dll) contain all the needed functions for your custom software development and integration of Elveflow instruments.

Since the source library is the same for each programming language (C++, MATLAB, and Python), the SDK functions are the same for each language and will be described only once in this guide. Please see the appropriate section for a complete description of all the available functions.

Due to their difference in operation, a description of the essential differences between each SDK's language will be described in a dedicated document. They will allow you to quickly grasp the specifics of each language and to start developing your custom software.



- Instruments are designated using their device name. The device name can be known and changed using National Instruments Measurement and Automation Explorer (NI MAX). The NI MAX Software should be automatically installed with Elveflow Smart Interface.
- The function "Check_Error" or "CheckError" is common for all the instruments. It is used to check errors from all functions, it uses LabVIEW errors that could be checked on the internet.
- An example function that could be used for feedback control is included in all libraries as an illustration only (see the specific [prototype](#)). It is provided as an example to help you create your own regulation system. Alternatively the remote mode, available for OB1, MSRD and BFS devices from V3_05_04, enables the library to handle the regulation, for the same device or between different ones. See the OB1 example file.

Description of SDK functions for each instrument:

OB1:

Normal workflow

The example “OB1_Ex__” illustrates the working principle of all the available SDK functions for the OB1.

The structure of the main program you would develop including Elveflow instruments should follow the same workflow as represented in the following figure. Using this workflow, you will start with a **configuration** and a **calibration** before starting to operate the OB1 and its connected sensors. Then, you can perform your instrumentation using the functions represented in the “**main working loop**”.

After the end of the operations, you **end the communication** by closing the communication with the OB1, clear the pointers and unload the DLL.

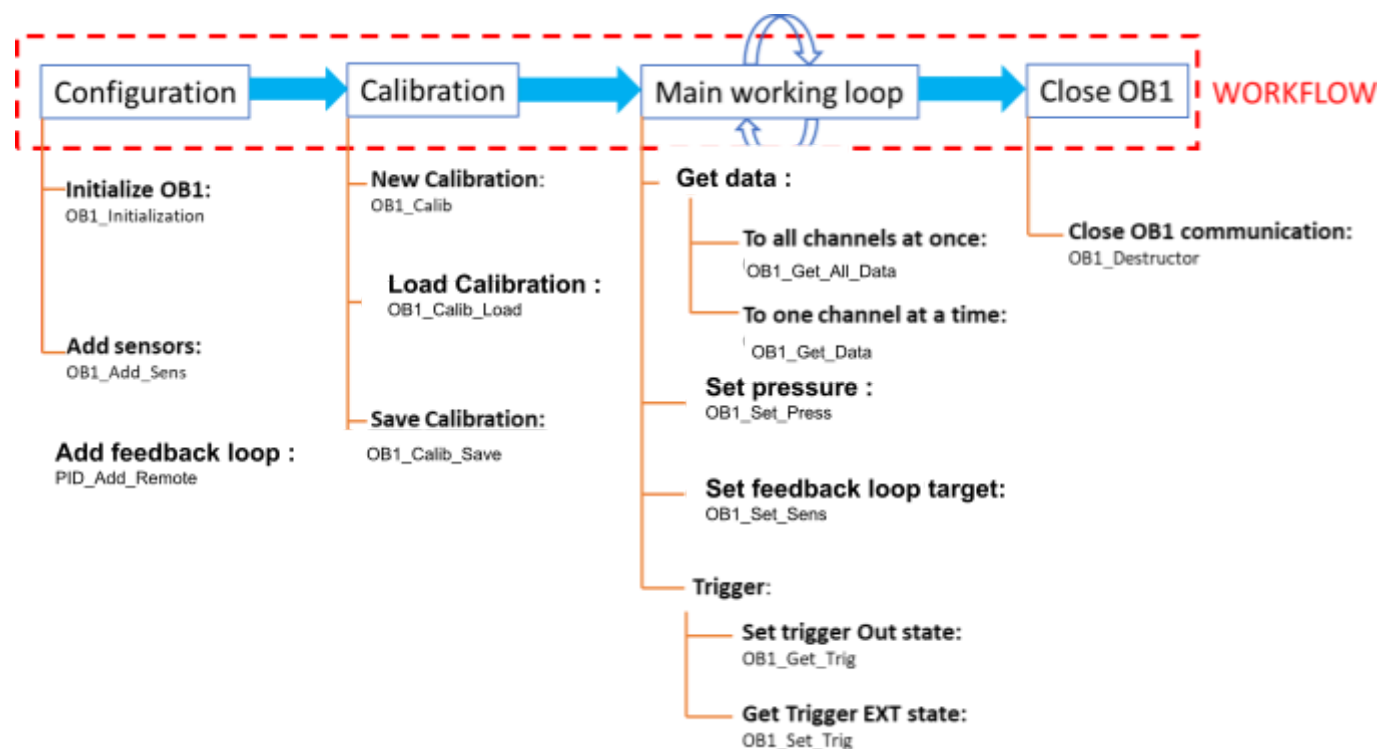


Figure 3 Typical workflow of a custom OB1 program representing the different types of the OB1 SDK functions

A description of each function can be found in the table below or in the form of script comments in the functions. To help debug the code, all functions will return an error code.

OB1_Initialization

Arguments :

- `char` : Device_Name[]
- `Z_regulator_type`: reg_ch_1
- `Z_regulator_type`: reg_ch_2
- `Z_regulator_type`: reg_ch_3
- `Z_regulator_type`: reg_ch_4
- `int32_t`: OB1_ID_out

Returns :

- `int32_t` : error constant (cf [Error handling](#))

Description :

Initialize the OB1 device using device name and regulator type (see SDK `Z_regulator_type` for corresponding numbers). It modifies the OB1 ID (number ≥ 0). This ID can be used with other functions to identify the targeted OB1. If an error occurs during the initialization process, the OB1 ID value will be -1.

Note for OB1 MK4 devices:

- the Device_Name is the VISA resource name, in the form of "COMX"
- The regulator type should be left at 0

OB1_Destructor

Arguments :

- `int32_t` : OB1_ID

Returns :

- `int32_t` : error constant (cf [Error handling](#))

Description :

Close communication with OB1

OB1_Add_Sens

Arguments :

- `int32_t` : OB1_ID
- `int32_t` : channel_1_to_4
- `Z_sensor_type` : sensor_type
- `Z_Sensor_digit_analog` : digital_or_analog
- `Z_Sensor_FSD_Calib` : fsens_digit_calib
- `Z_D_F_S_Resolution` : fsens_digit_resolution
- `double` : customsens_voltage_5_to_25

Returns :

- `int32_t` : error constant (cf [Error handling](#))

Description :

Add sensor to OB1 device. Select the channel n° (1-4) the sensor type. For the Flow sensor, the type of communication (Analog/Digital), the Calibration for digital version (H2O or IPA) should be specified as well as digital resolution (9 to 16 bits). (see SDK user guide, Z_sensor_type_type , Z_sensor_digit_analog, Z_Sensor_FSD_Calib and Z_D_F_S_Resolution for number correspondence).

For digital sensors, the sensor type is automatically detected during this function call.

For analog sensors, the calibration parameters are not taken into account.

If the sensor is not compatible with the OB1 version, or no digital sensor is detected an error will be thrown as output of the function.

OB1_Get_Data

Arguments :

- `int32_t` : OB1_ID
- `int32_t` : channel_1_to_4
- `double` : *regulator_data
- `double` : *sens_data

Returns :

- `int32_t` : error constant (cf [Error handling](#))

Description :

Read the requested channel, and return the pressure measured by the regulator and the sensor measurement attached to that channel.

OB1_Get_All_Data

Arguments :

- `int32_t` : OB1_ID
- `double` : *regulator_data channel 1
- `double` : *sens_data channel 1
- `double` : *regulator_data channel 2
- `double` : *sens_data channel 2
- `double` : *regulator_data channel 3
- `double` : *sens_data channel 3
- `double` : *regulator_data channel 4
- `double` : *sens_data channel 4

Returns :

- `int32_t` : error constant (cf [Error handling](#))

Description :

Return all pressure measurement from all regulators and all sensor measurements from all channels.

OB1_Set_Press

Arguments :

- `int32_t` : OB1_ID
- `int32_t` : channel_1_to_4
- `double` : pressure target

Returns :

- `int32_t` : error constant (cf [Error handling](#))

Description :

Set the pressure of the OB1 selected channel

OB1_Set_Sensor

Arguments :

- `int32_t` : OB1_ID
- `int32_t` : channel_1_to_4
- `double` : sens_data target

Returns :

- `int32_t` : error constant (cf [Error handling](#))

Description :

Start a PID feedback loop controlled by a given pressure channel of the selected OB1. A feedback loop needs to have been [configured first](#).

OB1_Get_Trig

Arguments :

- `int32_t` : OB1_ID
- `int32_t` : *trigger

Returns :

- `int32_t` : error constant (cf [Error handling](#))

Description :

Get the trigger of the OB1 (0 = 0V, 1 = 3,3V for MK3, 5V for MK4)

OB1_Set_Trig

Arguments :

- `int32_t` : OB1_ID

Returns :

- `int32_t` : error constant (cf [Error handling](#))
-

- `int32_t`: trigger

Description :

Set the trigger of the OB1 (0 = 0V, 1 = 3.3V, 5V for MK4)

OB1_Calib

Arguments :

- `int32_t`: OB1_ID

Returns :

- `int32_t`: error constant (cf [Error handling](#))

Description :

Launch OB1 calibration and return the calibration array. Before Calibration, ensure that ALL channels are properly closed with adequate caps.

OB1_Reset_Digit_Sens (OB1 MK3+)

Arguments :

- `int32_t`: OB1_ID
- `int32_t`: channel_1_to_4

Returns :

- `int32_t`: error constant (cf [Error handling](#))

Description :

Reset the digital sensor

OB1_Reset_Instr (OB1 MK3+)

Arguments :

- `int32_t`: OB1_ID

Returns :

- `int32_t`: error constant (cf [Error handling](#))

Description :

Reset the instrument

OB1_Calib_Load

Arguments :

- `int32_t` : OB1_ID
- `char` : path[]

Returns :

- `int32_t` : error constant (cf [Error handling](#))

Description :

Load the calibration file located at Path in the OB1 device. The function asks the user to choose the path if Path is not valid, empty or not a path.

OB1_Calib_Save

Arguments :

- `int32_t` : OB1_ID
- `char` : path[]

Returns :

- `int32_t` : error constant (cf [Error handling](#))

Description :

Save the Calibration cluster in the file located at Path. len is the Calib_Array_in array length. The function prompts the user to choose the path if Path is not valid, empty or not a path.

Elveflow_EXAMPLE_PID

Arguments :

- `int32_t` : PID_ID_in
- `double` : actualValue
- `int32_t` : Reset
- `double` : p
- `double` : i
- `int32_t` : *PID_ID_out
- `double` : *value

Returns :

- `int32_t` : error constant (cf [Error handling](#))

Description :

This function is only provided for illustration purposes, to explain how to do your own feedback loop. Elveflow does not guarantee neither efficient nor optimum regulation with this illustration of PI regulator.

With this function the PI parameters have to be tuned for every regulator and every microfluidic circuit. This function needs to be initiated with a first call where PID_ID = -1. The PID_out will provide the newly created PID_ID. This ID should be used in further calls. General remarks of this PI regulator : The error "e" is calculated for every step as e=target value-actual value. There are 2 contributions to a PI regulator: proportional contribution which only depend on this step and Prop=eP and integral part which is the "memory" of the regulator. This value is calculated as Integ=integral(ledt) and can be reset.

MSRD:

All the available functions for the programming of a customized program are detailed in the example "M_S_R_D_Ex_" contained in the appropriate example folder.

M_S_R_D_Initialization

Arguments :

- `char`: Device_Name[]
- `Z_sensor_type`: Sens_Ch_1
- `Z_sensor_type`: Sens_Ch_2
- `Z_sensor_type`: Sens_Ch_3
- `Z_sensor_type`: Sens_Ch_4
- `double` : CustomSens_Voltage_Ch12
- `double` : CustomSens_Voltage_Ch34
- `int32_t` : *MSRD_ID_out

Returns :

- `int32_t` : error constant (cf [Error handling](#))

Description :

Initialize the Sensor Reader device able to read digital sensors (MSRD) using device name and sensor type (see SDK `Z_sensor_type` for corresponding numbers). It modifies the MSRD ID (number ≥ 0). This ID can be used with other functions to identify the targeted MSRD. If an error occurs during the initialization process, the MSRD ID value will be -1. Initiate the communication with the Sensor Reader able to read digital sensors (MSRD). This VI generates an identification cluster of the instrument to be used with other VIs. NB: Sensor type has to be written here in addition to the "Add_Sens". NB 2: Sensors connected to channel 1-2 and 3-4 have to be the same type otherwise they will not be taken into account.

M_S_R_D_Destructor

Arguments :

- `int32_t` : M_S_R_D_ID

Returns :

- `int32_t` : error constant (cf [Error handling](#))

Description :

Close communication with MSRD.

M_S_R_D_Add_Sens

Arguments :

- `int32_t`: M_S_R_D_ID
- `int32_t`: channel_1_to_4
- `Z_sensor_type`: sensor_type
- `Z_Sensor_digit_analog`: digital_or_analog
- `Z_Sensor_FSD_Calib`: fsens_digit_calib
- `Z_D_F_S_Resolution`: fsens_digit_resolution
- `float32_t`: custom_sensor_voltage

Returns :

- `int32_t`: error constant (cf [Error handling](#))

Description :

Add sensor to MSRD device. Select the channel n° (1-4) the sensor type. For the Flow sensor, the type of communication (Analog/Digital), the Calibration for digital version (H2O or IPA) should be specified as well as digital resolution (9 to 16 bits). (see SDK user guide, `Z_sensor_type`, `Z_sensor_digit_analog`, `Z_Sensor_FSD_Calib` and `Z_D_F_S_Resolution` for the table of correspondence) For digital versions, the sensor type is automatically detected during this function call. For the Analog sensor, the calibration parameters are not taken into account. If the sensor is not compatible with the MSRD version, or no digital sensor is detected, an error will be thrown as output of the function. NB: Sensor type has to be the same as in the "Initialization" step.

M_S_R_D_Get_Data

Arguments :

- `int32_t`: M_S_R_D_ID
- `int32_t`: channel_1_to_4
- `double`: *sens_data

Returns :

- `int32_t`: error constant (cf [Error handling](#))

Description :

Read the sensor of the requested channel.s Units: Flow sensor: µl/min Pressure: mbar NB: For Digital Flow Sensor, If the connection is lost, MSRD will be reset and the return value will be zero.

M_S_R_D_Get_Trig (MSR V3)

Arguments :

- `int32_t`: M_S_R_D_ID
- `int32_t`: *trigger

Returns :

- `int32_t`: error constant (cf [Error handling](#))

Description :

Get the trigger of the M_S_R_D_ID (0 = 0V, 1 =5V).

M_S_R_D_Set_Trig (MSR V3)

Arguments :

- `int32_t` : M_S_R_D_ID
- `int32_t` : trigger

Returns :

- `int32_t` : error constant (cf [Error handling](#))

Description :

Set the trigger of the M_S_R_D_ID (0 = 0V, 1 =5V).

M_S_R_D_Set_Filt

Arguments :

- `int32_t` : M_S_R_D_ID
- `int32_t` : channel_1_to_4
- `LVBoolean` : ONOFF

Returns :

- `int32_t` : error constant (cf [Error handling](#))

Description :

Set filter for the corresponding channel.

M_S_R_D_Reset_Instr (MSR V2)

Arguments :

- `int32_t` : M_S_R_D_ID

Returns :

- `int32_t` : error constant (cf [Error handling](#))

Description :

Reset the MSRD device.

M_S_R_D_Reset_Sens (MSR V2)

Arguments :

- `int32_t` : M_S_R_D_ID

Returns :

- `int32_t` : error constant (cf [Error handling](#))

Description :

Reset the digital sensors connected to the MSRD.

BFS:

Please see the example file “_BFS_Example.cpp” for a standard usage of the available BFS functions. As with other instruments, there are three steps for programming: Initialization, instrumentation and resource liberation. Please note that for this particular sensor, in order to measure the flow rate ($\mu\text{L}/\text{min}$), you must first measure the volumetric mass density (g/L). Please see the table below for a description of each function.

BFS_Initialization

Arguments :

- `char`: Visa_COM[]
- `int32_t`: *BFS_ID_out

Returns :

- `int32_t`: error constant (cf [Error handling](#))

Description :

Initiate the BFS device using device com port (ASRLXXX::INSTR where XXX is the com port that could be found in windows device manager). It returns the BFS ID (number ≥ 0) to be used with other functions.

Make a density measurement by default so that even a basic flow measurement returns something different than NaN. Also set the measurement flags so that BFS_Get_Sensor_Data function returns effective measurement of the given parameter : flow, density and temperature. If the flag is False, the measurement will return 0.0

BFS_Destructor

Arguments :

- `int32_t`: BFS_ID_in

Returns :

- `int32_t`: error constant (cf [Error handling](#))

Description :

Close Communication with BFS device

BFS_Get_Data

Arguments :

- `int32_t`: BFS_ID_in
- `double`: *flow
- `double`: *density
- `double`: *temperature

Returns :

- `int32_t`: error constant (cf [Error handling](#))

Description :

Measure the fluid flow in (microL/min). The density can either be measured only once at the beginning of the experiment (ensure that the fluid flows through the sensor prior to density measurement), or before every flow measurement if the density might change. If you get +inf or -inf, the density wasn't correctly measured.

BFS_Zero

Arguments :

- `int32_t`: BFS_ID_in

Returns :

- `int32_t`: error constant (cf [Error handling](#))

Description :

Perform zero calibration of the BFS. Ensure that there is no flow when performed; it is advised to use valves. The calibration procedure is finished when the green LED stops blinking.

BFS_Set_Filter

Arguments :

- `int32_t`: BFS_ID_in
- `double`: *filter_value

Returns :

- `int32_t`: error constant (cf [Error handling](#))

Description :

Elveflow Library BFS Device Set the instrument Filter. 0.000001= maximum filter -> slow change but very low noise. 1= no filter -> fast change but noisy. Default value is 0.1

BFS_Set_Params

Arguments :

- `int32_t`: BFS_ID_in
- `double`: filter_value
- `boolean`: measure_temperature
- `boolean`: measure_density

Returns :

- `int32_t`: error constant (cf [Error handling](#))

Description :

Elveflow Library BFS Device Set the measurement flags so that BFS_Ge_Data function returns effective measurement of the given parameter : density and temperature. If the flag is False, the measurement will return 0.0.

MUX D-R-I (DISTRIBUTION, DISTRIBUTOR, RECIRCULATION or INJECTION):

Please see the example file “MUX_DRI_Ex_” for a standard usage of the available MUX DRI functions. The following table gives a description of the MUX DRI functions.

MUX_DRI_Initialization

Arguments :

- `char` : Visa_COM[]
- `int32_t` : *MUX_DRI_ID_out

Returns :

- `int32_t` : error constant (cf [Error handling](#))

Description :

Initiate the MUX Distribution, Distributor, Recirculation or Injection device using device COM port (ASRLXXX::INSTR where XXX is usually the COM port that could be found in Windows device manager). It returns the MUX D-R-I ID (number >=0) to be used with other functions.

MUX_DRI_Destructor

Arguments :

- `int32_t` : MUX_DRI_ID_in

Returns :

- `int32_t` : error constant (cf [Error handling](#))

Description :

Close Communication with MUX Distribution, Distributor, Recirculation or Injection device.

MUX_DRI_Set_Valve

Arguments :

- `int32_t` : MUX_DRI_ID_in
- `int32_t` : selected_Valve
- `Z_MUX_DRI_Rotation` : Rotation

Returns :

- `int32_t` : error constant (cf [Error handling](#))

Description :

Switch the MUX Distribution, Distributor, Recirculation or Injection to the desired valve. For MUX Distribution 12, between 1-12. For MUX Distributor (6 or 10 valves), between 1-6 or 1-10. For MUX Recirculation 6 or MUX Injection (6 valves), the two

states are 1 or 2. Rotation indicates the path the valve will perform to select a valve, either shortest 0, clockwise 1 or counterclockwise 2.

MUX_DRI_Get_Valve

Arguments :

- `int32_t`: MUX_DRI_ID_in
- `int32_t`: *selected_Valve

Returns :

- `int32_t`: error constant (cf [Error handling](#))

Description :

Get the current valve number. If the valve is changing, the function returns 0.

MUX_DRI_Send_Command (MUX Distribution 12 & MUX Recirculation 6)

Arguments :

- `int32_t`: MUX_DRI_ID_in
- `Z_MUX_DRI_Action`: action
- `char`: answer[]
- `int32_t`: length

Returns :

- `int32_t`: error constant (cf [Error handling](#))

Description :

Get the Serial Number or Home the valve. len is the length of the Answer. Remember that Home the valve takes several seconds. Home the valve is necessary as an initialization step before using the valve for a session.

Other MUX Series:

The MUX Series encompasses three instruments: MUX CROSS CHIP, MUX FLOW SWITCH and MUX WIRE. They are grouped together here because they use the same functions to start and end the communication. The table below gives the description of each function. The example “MUX_Ex__” illustrates the usage of all these functions.

MUX_Initialization

Arguments :

- `char` : Device_Name[]
- `int32_t` : *MUX_ID_out

Returns :

- `int32_t` : error constant (cf [Error handling](#))

Description :

Initiate the MUX device using device name (could be obtained in NI MAX). It return the F_S_R ID (number >=0) to be used with other functions.

MUX_Destructor

Arguments :

- `int32_t` : MUX_ID_in

Returns :

- `int32_t` : error constant (cf [Error handling](#))

Description :

Close the communication of the MUX device.

MUX_Get_Trig

Arguments :

- `int32_t` : MUX_ID_in
- `int32_t` : *Trigger

Returns :

- `int32_t` : error constant (cf [Error handling](#))

Description :

Get the trigger of the MUX device (0=0V, 1=5V).

MUX_Set_Trig

Arguments :

- `int32_t` : MUX_ID_in
- `int32_t` : Trigger

Returns :

- `int32_t` : error constant (cf [Error handling](#))

Description :

Set the Trigger of the MUX device (0=0V, 1=5V).

MUX_Set_all_valves (MUX FLOW SWITCH)

Arguments :

- `int32_t` : MUX_ID_in
- `int32_t` : array_valve_in[]
- `int32_t` : length

Returns :

- `int32_t` : error constant (cf [Error handling](#))

Description :

Valves are set by an array of 16 elements. If the valve value is equal to or below 0, the valve is closed, if it's equal to or above 1 the valve is open. The index in the array indicates the selected valve as shown below:

```

0 1 2 3
4 5 6 7
8 9 10 11
12 13 14 15
    
```

If the array does not contain exactly 16 elements nothing happens.

MUX_Set_indiv_valve(MUX CROSS CHIP)

Arguments :

- `int32_t` : MUX_ID_in
- `int32_t` : Input
- `int32_t` : Output
- `int32_t` : OpenClose

Returns :

- `int32_t` : error constant (cf [Error handling](#))

Description :

Set the state of one valve of the instrument. The desired valve is addressed using Input and Output parameters which correspond to the fluidics inputs and outputs of the instrument.

MUX_Wire_Set_all_valves (MUX WIRE instrument)

Arguments :

- `int32_t` : MUX_ID_in
- `int32_t` : array_valve_in[]
- `int32_t` : length

Returns :

- `int32_t` : error constant (cf [Error handling](#))

Description :

Valves are set by an array of 16 elements. If the valve value is equal to or below 0, the valve is closed, if it's equal to or above 1 the valve is open. If the array does not contain exactly 16 elements nothing happens.

MUX_Set_valves_Type (MUX WIRE V3 instrument)

Arguments :

- `int32_t` : MUX_ID_in
- `int32_t` : valveNb
- `int32_t` : type

Returns :

- `int32_t` : error constant (cf [Error handling](#))

Description :

Set the valve type. This function is available for MUX Wire V3 using custom Valves or Valve V2. Valve V3 type are automatically recognized by the MUX.

Custom types availables:

- 0 : Undefined (or delete custom valve)
- 4 : 2/2 Normally Closed Custom
- 5 : 2/2 Normally Opened Custom
- 6 : 3/2 Universal Custom

MUX_Get_valves_Type (MUX WIRE V3 instrument)

Arguments :

- `int32_t` : MUX_ID_in
- `int32_t` : types_array[]
- `int32_t` : length

Returns :

- `int32_t` : error constant (cf [Error handling](#))

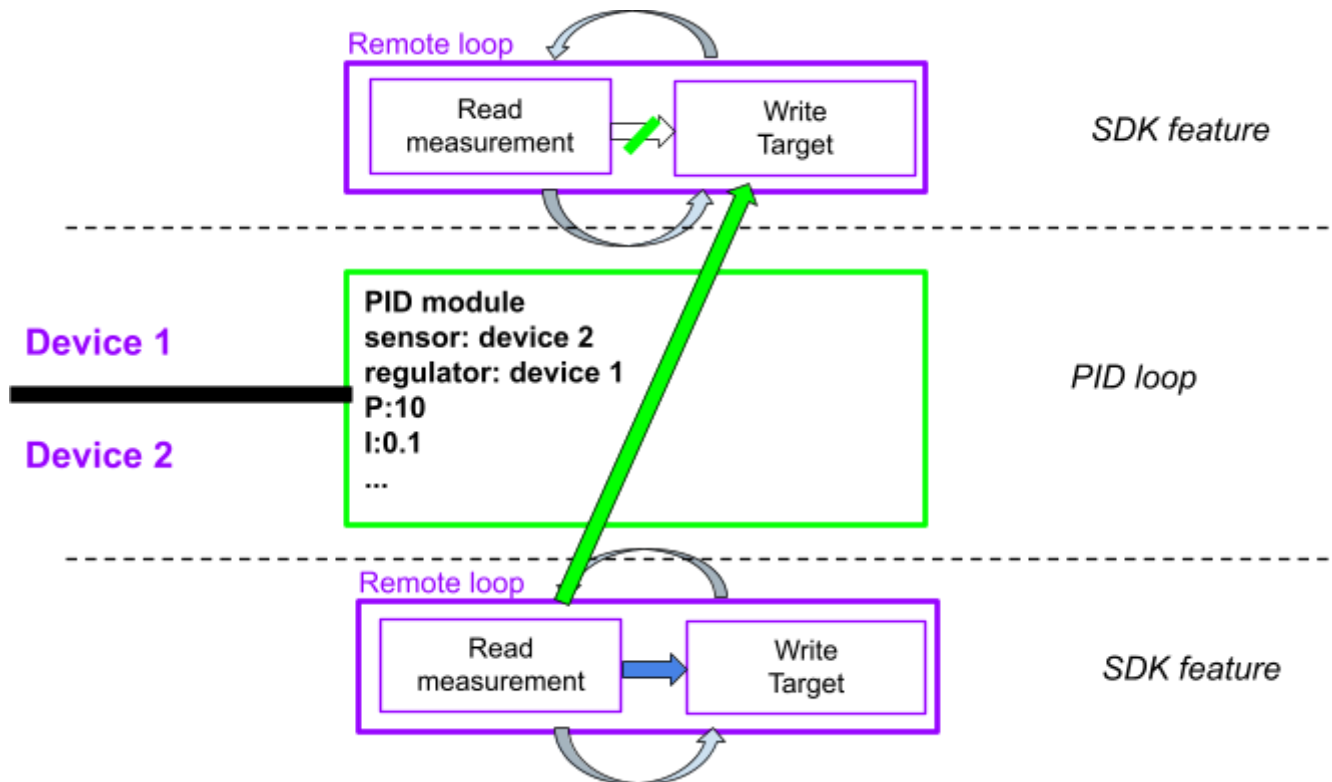
Description :

Read the Types values for all your MUX Wire V3 ports :

UNDEFINED = 0
2/2 Normally Closed = 1
2/2 Normally Opened = 2
3/2 Universal = 3
2/2 Normally Closed Custom = 4
2/2 Normally Opened Custom = 5
3/2 Universal Custom = 6

Remote PID:

If you configure any of the compatible instruments (OB1, MSRD, BFS), because the acquisition is run autonomously, you will also be able to configure, start and stop PID loops between these instruments without having to write the code itself of the loop. A fully working flow regulation can be started with an OB1 and an MFS with a single function call. Subsequent modification of the PID target is achieved in the remote loop of the device controlling the pressure/flow. A PID loop can be started on a single remote loop if the device can regulate the pressure/flow and a sensor is also connected to it.



In this figure, we consider the example of two devices both capable of pressure control and sensor reading. These two devices are configured in remote mode and therefore measure the sensor, then update the pressure automatically. By calling `add_PID` between device 1 and device 2, the measurement from the sensor of device 2 is broadcasted to the remote loop of device 1. A PID loop is then continuously running across the two remote loops and can be stopped, modified or reset on demand.

PID loops are destroyed when the device containing the regulator of the loop is closed.



PI or PID? The current SDK only allows PI parameters at the moment. Users who need to use the derivative term can use their own PID loop instead.

PID_Add_Remote

Arguments :

- `int32_t` : Regulator ID
- `int32_t` : Regulator Channel
- `int32_t` : Sensor ID
- `int32_t` : Sensor Channel
- `double` : Proportional parameter
- `double` : Integral parameter
- `int32_t` : Run (1) or stop (0) the PID

Returns :

- `int32_t` : error constant (cf [Error handling](#))

Description :

Add a PID loop between a regulator and a sensor. The PID loop can later be called with the device hosting the regulator coupled with its channel (if the device has more than 1). Only works when using the remote mode for the device(s) involved

PID_Set_Running_Remote

Arguments :

- `int32_t` : Regulator ID
- `int32_t` : Regulator Channel
- `int32_t` : Run (1) or stop (0) the PID

Returns :

- `int32_t` : error constant (cf [Error handling](#))

Description :

Adjust the running status of a PID loop. The PID loop is chosen based on the input device hosting the regulator coupled with the regulator channel (if the device has more than 1). Only works when using the remote mode for the device(s) involved.

PID_Set_Params_Remote

Arguments :

- `int32_t` : Regulator ID
- `int32_t` : Regulator Channel
- `int32_t` : (1) to reset the accumulated error value
- `double` : Proportional parameter
- `double` : Integral parameter

Returns :

- `int32_t` : error constant (cf [Error handling](#))

Description :

Adjust the PID parameters of a PID loop. The PID loop is chosen based on the input device hosting the regulator coupled with the regulator channel (if the device has more than 1). Only works when using the remote mode for the device(s) involved.

Appendix

Error handling:

All functions return an error code. If this code is 0 no error occurs. Other values indicate that an error occurs. Some personalized errors were added.

Error code:	Signification:
-8000	No Digital Sensor found
-8001	No pressure sensor compatible with OB1 MK3
-8002	No Digital pressure sensor compatible with OB1 MK3+
-8003	No Digital Flow sensor compatible with OB1 MK3
-8004	No IPA config for this sensor
-8005	Sensor not compatible
-8006	No Instrument with selected ID
-8007	Wrong MUX device
-8008	Only available for MUX Wire V3 devices
-8009	Type 1,2 and 3 are reserved for V3 valves. If you are using custom or older valves please use 4,5 and 6 types
-8030	No communication with OB1
-8031	No communication with BFS
-8032	No communication with MSRD
-8033	OB1 remote loop has not been executed
-8034	BFS remote loop has not been executed
-8035	MSRD remote loop has not been executed

Other errors can be found in the LabVIEW error user guide. (<http://www.ni.com/pdf/manuals/321551a.pdf>)

List of constants, prototypes and description (for C++, MATLAB and Python):

Z_sensor_type_Level stands for all types of level sensor such as bubble detectors.

Constants (define Elveflow.h as uint16_t):

Z_regulator_type:

Z_regulator_type_none	0
Z_regulator_type__0_200_mbar	1
Z_regulator_type__0_2000_mbar	2
Z_regulator_type__0_8000_mbar	3
Z_regulator_type_m1000_1000_mbar	4
Z_regulator_type_m1000_6000_mbar	5

(Note : set to 0 for OB1 MK4 devices)

Z_Sensor_digit_analog:

Z_Sensor_digit_analog_Analog	0
Z_Sensor_digit_analog_Digital	1

Z_D_F_S_Resolution:

Z_D_F_S_Resolution__9Bit	0
Z_D_F_S_Resolution__10Bit	1
Z_D_F_S_Resolution__11Bit	2
Z_D_F_S_Resolution__12Bit	3
Z_D_F_S_Resolution__13Bit	4
Z_D_F_S_Resolution__14Bit	5
Z_D_F_S_Resolution__15Bit	6
Z_D_F_S_Resolution__16Bit	7

Z_MUX_DRI_Rotation:

Z_MUX_DRI_Rotation_Shortest	0
Z_MUX_DRI_Rotation_Clockwise	1
Z_MUX_DRI_Rotation_CounterClockwise	2

Z_sensor_type:

Z_sensor_type_none	0
Z_sensor_type_Flow_1_5_muL_min	1
Z_sensor_type_Flow_7_muL_min	2
Z_sensor_type_Flow_50_muL_min	3
Z_sensor_type_Flow_80_muL_min	4
Z_sensor_type_Flow_1000_muL_min	5
Z_sensor_type_Flow_5000_muL_min	6
Z_sensor_type_Press_70_mbar	7
Z_sensor_type_Press_340_mbar	8
Z_sensor_type_Press_1_bar	9
Z_sensor_type_Press_2_bar	10
Z_sensor_type_Press_7_bar	11
Z_sensor_type_Press_16_bar	12
Z_sensor_type_Level	13
Z_sensor_type_Custom	14
Z_sensor_type_Flow_40000_muL_min	15

Z_Sensor_FSD_Calib:

Z_Sensor_FSD_Calib_H2O	0
Z_Sensor_FSD_Calib_IPA	1

Z_MUX_DRI_Action:

Z_MUX_DRI_Action_Home	0
Z_MUX_DRI_Action_SerialNumber	1